Table des matières

Cours 07 — ELT en régime permanent : incrémental et orchestration 1
Objectifs 2
1. Le chantier et le régime permanent 3
2. Une source qui change de forme 4
3. La matérialisation incrémentale 4
4. Sur quoi filtrer : l'arrivée, jamais l'événement 5
5. La fenêtre de reprise 6
6. Tout refaire, et ce qui ne se refait pas 8
7. Ce que l'incrémental ne sait pas faire 8
8. Une dimension conforme qui se rouvre 9
9. Deux environnements 10
10. L'orchestration, du bon côté de la frontière 11
11. Ce que trois mois changent 11
Points clés à retenir 12
Exercices 13

Cours 07 — ELT en régime permanent : incrémental et orchestration

Théorie · Module 7/9 — Formation Analytics Engineering · intermédiaire

Où vous êtes : `RAW` → `STAGING` → `INTERMEDIATE` → `MARTS` → `PRESENTATION`. Les cinq couches sont debout, et le projet se reconstruit d'un trait. Ce cours ne change pas leur ordre : il fait tourner la chaîne en boucle, sur un lot qui revient chaque mois.

En 30 secondes :

- Un pipeline monté une fois et un pipeline rejoué ne posent pas les mêmes problèmes. Le second demande qu'on dise ce qui est neuf, et qu'on sache le redire après une panne.
- Un modèle incrémental ne recalcule que ce qu'un filtre laisse passer. Trois pièces le font : `is_incremental()`, `unique_key` et `{{ this }}`.
- Le filtre porte sur la date d'arrivée, jamais sur la date de l'événement. Un filtre sur l'événement perd les retardataires, en silence et pour toujours.

© 2026 iAlexMG. Tous droits réservés.

- Une fenêtre de reprise relit quelques jours à chaque passe. Elle coûte peu et elle assure une propriété que rien n'impose à la source.
- La clé unique est ce qui rend la relecture inoffensive : une ligne relue remplace la sienne.
- Une source semi-structurée change de forme sans prévenir. Un champ renommé entre deux versions de micrologiciel se réunit au `STAGING`, à un seul endroit.
- Un fait d'accumulation ne se charge pas par ajout : ses lignes changent après coup.
- Deux environnements, un seul dépôt. C'est `--target` qui décide où on écrit, pas le code.
- Trois mois de données de plus transforment une cohorte en verdict. C'est le vrai produit d'un entrepôt qui tourne.

L'idée du cours. Jusqu'ici, Boréal a bâti un entrepôt sur un historique figé, du 1er janvier 2022 au 31 décembre 2025. Un entrepôt réel ne vit pas ainsi : chaque mois, l'ERP recrache un extrait, les garanties livrent leurs réclamations, et le courtier de télémétrie dépose une archive par jour. Ce cours fait passer le projet du chantier au régime permanent. Vous chargez trois mois de 2026, vous rendez incrémental le seul fait qui le mérite, et vous découvrez au passage que la question difficile n'est pas « comment n'ajouter que le neuf » mais « qu'est-ce que le neuf, au juste ». À la fin, l'incident du compresseur d'air que le Cours 06 avait laissé sans verdict en a un.

Objectifs

- Dire ce qui change quand un pipeline passe du chantier au régime permanent.
- Aplatir une source semi-structurée qui porte un objet imbriqué, une liste et un champ renommé.
- Réunir deux noms d'un même champ à l'endroit unique qui doit le faire.
- Écrire un modèle incrémental, et nommer le rôle de chacune de ses trois pièces.
- Choisir la colonne sur laquelle un filtre incrémental doit porter, et défendre ce choix.
- Reconnaître ce qu'un filtre sur la date de l'événement perd, et le chiffrer.
- Poser une fenêtre de reprise, et dire ce qu'elle coûte et ce qu'elle assure.
- Dire ce que `unique_key` apporte que le filtre n'apporte pas.
- Reconnaître un modèle qu'il ne faut pas rendre incrémental.
- Dire ce que `--full-refresh` fait et ne fait pas, et vérifier une croyance sur un outil.
- Séparer développement et production par une cible et une macro.

© 2026 iAlexMG. Tous droits réservés.

- Situer l'orchestration du bon côté de la frontière entre les deux métiers.
- Lire une cohorte identifiée trois mois plus tôt, et dire ce que les nouvelles données prouvent.

---

1. Le chantier et le régime permanent

Labo : `Labo_07_AE.md`, étapes 1, 2, 7 et 11.

Un projet analytique connaît deux vies. La première est un chantier : on charge un historique, on bâtit des modèles, on ajuste, et chaque exécution repart de zéro parce que c'est le plus simple et que rien ne coûte cher. La seconde commence le jour où le pipeline est planifié et où quelqu'un d'autre lit ce qu'il produit.

Chez Boréal, la cadence est celle des sources, et elles n'ont pas la même.

La source	Ce qu'elle livre	À quel rythme
L'ERP	un extrait des tables du mois écoulé	la nuit qui suit la fin du mois
Les garanties	clients, enregistrements, visites, réclamations	en fin de mois
La télémétrie	une archive des messages reçus dans la journée	un fichier par jour
L'usine	l'historien du SCADA, relevé par relevé	en continu, livré par Data Engineering à l'heure

Le pipeline n'a pas à connaître ces rythmes : Data Engineering les absorbe et livre la `RAW`. Ce qui change pour vous est ailleurs, et tient en trois questions que le chantier ne posait pas.

Ce qui est neuf, d'abord. Tant qu'on reconstruit tout, la question ne se pose pas. Dès qu'on veut n'ajouter que le neuf, il faut une définition du neuf, et c'est le sujet des sections 4 et 5.

Ce qui a changé, ensuite. Un extrait mensuel ne porte pas que des lignes nouvelles : il porte aussi des lignes anciennes dans une version corrigée. Une réclamation soumise en novembre revient en janvier avec son paiement. Un pipeline qui ne fait qu'ajouter tiendrait pour vrai un état périmé.

Ce qu'on fait après une panne, enfin. Une exécution qui casse au milieu laisse l'entrepôt dans un état qu'il faut pouvoir reprendre sans réfléchir. C'est l'idempotence, et c'est l'exigence maîtresse du régime permanent : relancer une passe déjà passée ne doit rien changer au résultat. Un pipeline qu'on n'ose pas relancer est un pipeline cassé.

À retenir — le régime permanent ne se mesure pas au temps gagné. Chez Boréal, tout reconstruire prend cinq secondes, et l'incrémental n'en sauve pas trois. Ce qu'il apporte est ailleurs : il oblige à nommer la clé du grain et la définition du neuf, deux choses qu'un projet qui se refait en entier peut ignorer jusqu'au jour où il ne le peut plus.

© 2026 iAlexMG. Tous droits réservés.

2. Une source qui change de forme

Labo : `Labo_07_AE.md`, étapes 3 et 4.

La télémétrie est le sixième processus de la matrice de bus, et la dernière source du parcours à entrer dans l'entrepôt. C'est aussi la plus retorse, parce qu'elle n'a pas la forme d'un tableau.

Chaque véhicule connecté publie un résumé par sortie sur un courtier de messages. L'abonné de Boréal écrit ce qu'il reçoit dans une archive par jour, une ligne par message, et Data Engineering la charge telle quelle : une colonne `JSON` nommée `message`, plus l'heure de réception et le sujet de la publication. Trois difficultés y tiennent ensemble, et aucune n'est propre à Boréal.

Un objet imbriqué. La position du départ arrive sous `position`, avec `lat` et `lon` dessous. On y descend d'un cran à la fois, et le Cours 06 a déjà posé les deux opérateurs : `->` rend du JSON, `->>` rend du texte.

Une liste. Les codes d'anomalie levés pendant la sortie arrivent dans un tableau, vide la plupart du temps. Une liste n'est pas une colonne : elle se déplie en lignes, et la section 8 dit où.

Un champ qui change de nom. Le micrologiciel 2.0 du module télématique, diffusé à partir du 1er mai 2025, ajoute un champ et en renomme un autre. La température maximale du moteur s'appelle `temp_moteur_max` jusqu'à la version 1.6.2 et `temperature_moteur_max` à partir de la 2.0.1 ; les heures moteur n'existent qu'à partir de la 2.0.1. Sur l'archive complète, cela fait 87 374 messages d'un côté et 114 168 de l'autre, et aucun message ne porte les deux noms.

C'est ce qu'on appelle une dérive de schéma (*schema drift*), et c'est la règle plutôt que l'exception dès qu'une source est un appareil déployé chez des clients. Le parc ne se met pas à jour d'un coup, et les deux formes coexistent pendant des mois.

La parade est celle du `STAGING`, et elle n'a rien de neuf : la couche a pour seule responsabilité de rendre chaque source lisible, donc c'est là que les deux noms se réunissent, en une expression, une fois. Aucun modèle en aval ne doit savoir qu'il y a eu deux versions.

Piège — réunir deux champs ailleurs qu'au `STAGING`. La tentation est de traiter le cas dans le rapport qui s'en plaint, parce que c'est là qu'on découvre le problème. Un an plus tard, le troisième rapport qui lit la même colonne est faux, et personne ne sait pourquoi les deux premiers ne le sont pas.

3. La matérialisation incrémentale

Labo : `Labo_07_AE.md`, étapes 5, 6 et 12.

Au Cours 04, les faits ont été matérialisés en `table` : chaque `dbt run` les reconstruit de zéro. La matérialisation `incremental` change ce comportement. Elle a deux régimes, et c'est dbt qui choisit lequel s'applique.

La situation	**Ce que dbt fait**
La table n'existe pas	il la crée en entier, exactement comme une `table`

© 2026 iAlexMG. Tous droits réservés.

La situation	Ce que dbt fait
La table existe	il n'exécute que ce que le filtre laisse passer, et fusionne le résultat

Trois pièces le rendent possible, et elles ne servent pas à la même chose. Les confondre est la source de presque toutes les erreurs de ce mécanisme.

`is_incremental()` est une fonction qui vaut faux à la première exécution et vrai ensuite. Elle sert à ouvrir un bloc qui n'a de sens que lorsque la table existe déjà, et ce bloc porte le filtre. Son rôle est la vitesse.

`{{ this }}` désigne la table qu'on est en train de bâtir. C'est la seule façon de lui demander où elle s'est arrêtée, et elle ne s'emploie qu'à l'intérieur du bloc précédent, faute de quoi elle désignerait une table qui n'existe pas encore.

`unique_key` déclare la clé du grain. Elle sert la justesse : une ligne dont la clé est déjà présente remplace la sienne au lieu de s'ajouter. C'est elle, et elle seule, qui rend une passe rejouable.

La stratégie dit comment la fusion se fait. Sur DuckDB, `delete+insert` supprime d'abord les lignes dont la clé revient, puis insère le lot. D'autres plateformes proposent `merge`, `append`, ou des stratégies propres à leur moteur ; le principe ne change pas.

À retenir — le filtre sert la vitesse, la clé sert la justesse. Un modèle incrémental sans `unique_key` accumule des doublons à la première relecture. Un modèle avec la clé et sans filtre est simplement lent. La faute grave est l'inverse de celle qu'on craint : c'est la clé qui manque, pas le filtre.

Reste à savoir quoi rendre incrémental. La règle tient en une phrase : ce qui est gros et qui ne fait que grossir. Les faits de transaction, donc, et pas les dimensions, qui sont petites et dont les lignes changent en place. Chez Boréal, un seul modèle le mérite aujourd'hui, et c'est le sixième fait : 201 542 sorties contre 80 618 lignes de vente et 4 337 réclamations.

4. Sur quoi filtrer : l'arrivée, jamais l'événement

Labo : `Labo_07_AE.md`, étapes 8 et 9.

Voici la question qui fait la valeur de ce cours, et l'exemple qu'on en donne d'ordinaire est le mauvais. Il ressemble à ceci :

```
{% if is_incremental() %}
where cle_date > (select max(cle_date) from {{ this }})
{% endif %}
```

Cela se lit bien : ne prendre que les événements postérieurs au dernier qu'on ait. Et c'est faux, parce qu'un événement et son arrivée dans l'entrepôt sont deux instants différents.

Chez Boréal, un module télématique hors de la couverture cellulaire garde ses messages et les publie au retour. Sur l'archive livrée, 8 333 messages arrivent plus d'une heure après la fin de leur sortie, et le retard maximal mesuré est de 96 heures. Le phénomène suit la saison : février, janvier et décembre en comptent le plus, 2 169, 1 772 et 1 281, ce qui est exactement ce qu'on attend d'un parc de motoneiges au fond d'une vallée.

© 2026 iAlexMG. Tous droits réservés.

Le filtre ci-dessus ne voit jamais ces messages. Leur date de sortie est antérieure au maximum déjà chargé, la condition les écarte, et aucune passe ultérieure ne les rattrapera puisque le maximum ne fait que monter. Ils sont perdus, en silence.

La mesure, sur les deux passes mensuelles que le labo fait faire : 47 sorties perdues, 2 558,2 kilomètres et 2 codes d'anomalie. Toujours les deux derniers jours du mois, et toujours définitivement.

Jour de sortie	Sorties perdues	Reçues le
2026-01-30	2	2026-02-01
2026-01-31	22	2026-02-01
2026-02-27	2	2026-03-01
2026-02-28	21	2026-03-01

Quarante-sept sur 201 542, c'est deux centièmes de pour cent, et on serait tenté de hausser les épaules. Ce serait passer à côté de deux choses. La perte n'est pas aléatoire : elle frappe systématiquement la fin de chaque période, donc exactement les lignes qu'un rapport mensuel regarde. Et elle dépend de la cadence : la même faute appliquée à toute l'archive perdrait 125 sorties au rythme mensuel, et 2 559 sorties pour 116 428,6 kilomètres au rythme quotidien, qui est celui de la source. Plus le pipeline tourne souvent, plus il perd.

Le bon filtre porte sur l'arrivée. Ici, c'est `recu_le`, l'heure à laquelle l'abonné a reçu le message. Un retardataire garde une date de sortie ancienne, mais sa date de réception est récente : le filtre le prend.

```
{% if is_incremental() %}
where recu_le > (select max(recu_le) from {{ this }})
{% endif %}
```

Rejoué sur les mêmes passes, ce filtre ne perd rien : 201 542 lignes des deux côtés, aucun doublon.

Piège — la colonne qui s'appelle « date » n'est presque jamais la bonne. Dans un fait, la colonne de date est celle que le métier lit, et c'est par construction celle de l'événement. La colonne du filtre est technique, elle ne sert à personne d'autre qu'au pipeline, et elle mérite d'être gardée dans le fait pour cette seule raison.

5. La fenêtre de reprise

Labo : `Labo_07_AE.md`, étape 10.

Le filtre sur l'arrivée est juste, et il repose sur une hypothèse : qu'un message n'arrive jamais avec une date de réception inférieure à une date déjà vue. Cette hypothèse tient chez Boréal, et on peut le vérifier plutôt que le croire. L'archive compte 933 jours, aucun n'a été livré deux fois, et les 2 943 messages livrés en double le sont toujours le même jour, à une minute d'écart au plus.

© 2026 iAlexMG. Tous droits réservés.

Rien n'impose cette propriété. Elle tient parce que l'abonné écrit son fichier une fois par jour et ne le rouvre pas, et le jour où l'équipe qui l'exploite changera cela, personne ne vous préviendra. C'est la définition d'une dette : un modèle juste pour une raison qui n'est écrite nulle part.

La fenêtre de reprise est l'assurance correspondante. Elle consiste à reculer le repère de quelques jours, et à relire cette tranche à chaque passe.

```
{% if is_incremental() %}
where recu_le > (select max(recu_le) - interval 3 day from {{ this }})
{% endif %}
```

Ce qu'elle coûte se mesure, et ce n'est pas rien.

La passe	Lignes neuves	Lignes relues
Lot de février 2026	29 843	2 199
Lot de mars 2026	12 918	2 772

Ce qu'elle rapporte est nul sur ce jeu de données, et il faut le dire ainsi : les passes avec fenêtre donnent exactement le même résultat que sans. Une assurance dont on n'a pas eu besoin n'est pas une assurance inutile, mais ce n'est pas non plus une démonstration, et un support qui ferait semblant du contraire vous apprendrait à choisir au hasard.

Ce qui justifie la fenêtre est la largeur qu'on lui donne, et elle se choisit sur une mesure : le plus long retard de livraison observé sur la source. Ici, 96 heures. Trois jours est donc une marge serrée, sept jours une marge confortable, et trente jours le signe qu'on n'a pas mesuré.

La relecture n'est inoffensive que grâce à `unique_key`. Sans elle, une fenêtre de trois jours ajoute trois jours de doublons à chaque passe, et le modèle devient pire que s'il n'y avait pas de fenêtre du tout : mesuré, une seule passe sans clé porte le fait à 202 330 lignes pour 201 542 messages distincts, soit les 788 lignes de la fenêtre comptées deux fois.

Une dernière chose, et c'est celle qu'il faut retenir de la section précédente autant que de celle-ci. Une fenêtre assez large répare un filtre posé sur la mauvaise colonne, et le fait sans rien dire : posée sur la date de l'événement, une fenêtre de trois jours retrouve les 47 sorties perdues de la section 4, mesuré sur les mêmes passes, pour un volume relu presque identique, 32 034 lignes contre 32 042 à la passe de février. Elle marche parce que la cadence mensuelle est bien plus large que le retard maximal de la source, et pour aucune autre raison. Resserrez la cadence à celle de l'archive, un jour, et la même fenêtre de trois jours laisse repasser 33 sorties ; il en faut sept pour n'en perdre aucune. Un filtre sur l'arrivée, lui, ne dépend pas de la cadence. La fenêtre est une assurance, pas une réparation, et une assurance qui répare un défaut de conception est le meilleur moyen de ne jamais le voir.

Pourquoi — relire est moins cher qu'on ne croit, et parfois pas moins cher du tout. Sur ce projet, la fenêtre ne change pas le temps d'exécution, parce que le modèle lit une vue de `STAGING` dont la déduplication emploie une fonction de fenêtre : le moteur ne peut pas pousser le filtre à travers, et il lit toute la source de toute façon. Mesuré : 0,22 seconde pour lire 788 lignes à travers la vue, 0,21 seconde pour en lire 201 542, et 0,013 seconde pour poser la même fenêtre directement sur la table source. L'incrémental économise l'écriture, pas la lecture. À ce volume cela n'a aucune importance, et une vue par source vaut mieux qu'une optimisation ; à cent fois le volume, c'est la première chose à regarder.

© 2026 iAlexMG. Tous droits réservés.

6. Tout refaire, et ce qui ne se refait pas

Labo : `Labo_07_AE.md`, étapes 11 et 14.

Un modèle incrémental accumule, et il ne revoit jamais le passé. Il faut donc une trappe de sortie pour le jour où la logique change : on a corrigé une règle du `STAGING`, ajouté une colonne, changé le décodage d'un code, et les anciennes lignes sont désormais fausses.

```
dbt build --full-refresh
```

`--full-refresh` dit à dbt d'ignorer le comportement incrémental pour cette exécution : il détruit la table et la rebâtit en entier, comme au premier jour. C'est la commande à lancer après toute retouche d'un modèle incrémental ou de ce qu'il lit, et elle est sans danger, parce qu'un modèle se recalcule depuis la `RAW`.

Elle est aussi le seul contrôle sérieux qu'on ait sur l'incrémental. Un chargement par tranches et un chargement d'un bloc doivent donner la même chose, ligne pour ligne, et il n'y a qu'une façon de le savoir : faire les deux et comparer. Chez Boréal, les quatre passes successives et un `--full-refresh` rendent les mêmes 201 542 lignes.

Une croyance répandue veut qu'un `--full-refresh` détruise l'histoire d'un instantané. Sur cette pile, c'est faux, et cela se vérifie en deux commandes. `dbt snapshot --full-refresh` est refusé par dbt 1.12.3, qui ne connaît pas cette option sur cette commande. Et `dbt build --full-refresh` exécute l'instantané comme d'habitude, sans le refaire : mesuré sur le projet complet, 205 versions et 49 fermées avant comme après, et les ventes de 2023 par palier d'époque inchangées au cent près.

Ce qui demande de l'attention n'est donc pas l'option, c'est l'ordre. La commande du projet reste `dbt build --exclude snp_concessionnaire` pour la raison que le Cours 05 a posée : une exécution nue lance l'instantané sans variable de lot, donc au présent, hors de la séquence dans laquelle son histoire se monte. Après les quatre lots, cela ne change rien ; avant, cela ouvre des versions datées d'aujourd'hui devant lesquelles tout lot ancien rejoué paraît fermé avant d'être ouvert.

À retenir — vérifier avant de répéter. La phrase « un full-refresh détruit un snapshot » circule, elle a été vraie sur d'anciennes versions, et elle est fausse ici. Une croyance sur le comportement d'un outil se vérifie en une commande ; c'est moins cher que de bâtir une procédure autour d'un danger qui n'existe pas.

7. Ce que l'incrémental ne sait pas faire

Labo : `Labo_07_AE.md`, étape 2.

L'incrémental suppose que le passé ne bouge pas. Trois des six faits de Boréal respectent cette règle, et un la viole ouvertement : le fait d'accumulation des réclamations, dont le Cours 06 a dit que sa ligne se met à jour au lieu de s'ajouter.

La mesure est nette. Sur les 3 348 réclamations que l'entrepôt portait au 31 décembre 2025, 154 changent avec les trois lots de 2026, soit 4,6 pour cent. Ce sont des dossiers qui ont reçu leur décision ou leur paiement pendant l'hiver : les statuts déclarés `SOUMISE` passent de 48 à 41, les `APPROUVEE` de 106 à 179, les `PAYEE` de 2 799 à 3 690.

© 2026 iAlexMG. Tous droits réservés.

Un modèle incrémental filtré sur la date d'ouverture tiendrait ces 154 lignes pour vraies, et rien ne le signalerait. Le détail qui achève l'argument : la somme des montants de ces 3 348 dossiers ne change pas d'un cent, 1 459 608,53 dollars avant comme après. Le montant est fixé à la soumission ; ce qui bouge, ce sont les jalons. Une réconciliation sur l'argent ne verrait donc rien.

Trois réponses existent, et le choix se fait selon la taille.

La réponse	Quand
Laisser le fait en `table`	il est petit : 4 337 lignes se refont en un dixième de seconde
Filtrer sur l'arrivée du lot, avec `unique_key`	il est gros, et la source redonne les lignes qui changent
Découper le fait en deux, l'ancien figé et le récent refait	il est très gros, et les vieux dossiers ne bougent plus

Chez Boréal, c'est la première, et il faut savoir la défendre : le bon usage de l'incrémental est de ne pas s'en servir quand rien ne l'exige.

8. Une dimension conforme qui se rouvre

Labo : `Labo_07_AE.md`, étapes 15, 16 et 17.

Le Cours 06 a laissé `dim_anomalie` à l'état de dimension dégénérée, portée par le fait des réclamations, et a annoncé que la question se rouvrirait ici. Elle se rouvre parce qu'un second fait porte le même code.

Les codes d'anomalie arrivent maintenant par deux chemins qui ne se parlent pas : le technicien qui lit le calculateur en atelier et le reporte dans la réclamation, et le module télématique qui publie ce qu'il a levé pendant la sortie. Dès qu'une valeur sert de clé à deux faits, la question n'est plus « faut-il une dimension » mais « comment deux rapports pourraient-ils s'en passer ».

Le déplier suppose un fait de plus, parce qu'une sortie peut lever plusieurs codes : 1 155 codes sur 1 140 sorties. Le grain n'est donc pas celui de `fct_sorties`, et la forme est celle que le Cours 06 a posée, le fait sans mesure. Une ligne par sortie et par code, aucune quantité, l'information étant que la ligne existe.

Ce que la dimension fait voir se lit en une requête, et c'est ce qui justifie son existence.

Le code	Véhicules réclamés	Véhicules qui l'ont levé en roulant
`D240`, patinage de la courroie	239	500
`E120`, tension de batterie basse	418	155
`H510`, pression d'huile basse	192	103
`T310`, surchauffe du moteur	174	96

© 2026 iAlexMG. Tous droits réservés.

Le code	Véhicules réclamés	Véhicules qui l'ont levé en roulant
`T320`, niveau de liquide bas	0	211

La dernière ligne est le résultat. Deux cent onze véhicules ont signalé un niveau de liquide de refroidissement bas en roulant, et pas une réclamation ne porte ce code. Sans la dimension, ce zéro n'a nulle part où s'afficher : il n'existe que comme absence, et une absence ne sort d'aucune requête qui ne la cherche pas.

Les libellés, eux, ne vivent dans aucune table. Ils sont dans un guide du portail des concessionnaires, `GUI-CODES`, version 1 du 3 juin 2024. C'est le cas d'usage d'un `seed` : un référentiel court, stable, qu'aucun système ne livre, versionné avec le code plutôt que chargé depuis une source. `dbt seed` lit les CSV du dossier `seeds/` et en fait des tables.

Piège — le `seed` qui grossit. Un `seed` est une bonne réponse à cinq lignes et une mauvaise à cinq mille. Dès qu'un référentiel change assez souvent pour qu'on veuille savoir qui l'a modifié et quand, il lui faut une source et un propriétaire, pas un fichier dans le dépôt des modèles.

9. Deux environnements

Labo : `Labo_07_AE.md`, étape 13.

Jusqu'ici, tout votre travail atterrissait au même endroit, et l'entrepôt que vous bricolez est celui que le rapport lit. En régime permanent, c'est intenable : un modèle à moitié réécrit, et le tableau de bord de la direction est faux pendant une heure.

dbt sépare les deux par les cibles (*targets*) du `profiles.yml`. Un bloc par environnement sous `outputs:`, et `--target` choisit à l'exécution :

```
dbt build                # la cible par défaut, dev
dbt build --target prod  # la production
```

Il reste à faire écrire chaque cible ailleurs, et c'est la seule retouche du parcours à la macro `generate_schema_name` du Cours 03 : en production, les noms nus, `staging` et `marts` ; partout ailleurs, les mêmes préfixés de `dev_`.

`target.name`	**Un modèle qui déclare `+schema: marts` écrit dans…**
`prod`	`marts`
`dev`	`dev_marts`

Un seul dépôt, un seul jeu de modèles, et rien dans le code ne dit dans quel environnement il tourne. C'est le point, et c'est ce qui rend la production reproductible : ce qui a été éprouvé en développement est exactement ce qui s'exécute.

Astuce — ce que dev et prod partagent, et ce qu'ils ne devraient pas partager. Ici, les deux cibles visent le même fichier DuckDB et ne se distinguent que par le préfixe des schémas, ce qui suffit pour un projet solo et se lit bien. En entreprise, la production a sa propre base et un compte de service distinct, de sorte qu'une fausse manœuvre en développement ne puisse pas l'atteindre. Sur DuckDB, l'équivalent serait un second fichier, et le `profiles.yml` le dirait par un autre `path`.

© 2026 iAlexMG. Tous droits réservés.

10. L’orchestration, du bon côté de la frontière

Un pipeline en production ne se lance pas à la main tous les mois. Orchestrer, c’est répondre à trois questions : quoi lancer, dans quel ordre, et quand.

Les deux premières sont déjà réglées, et c’est la bonne nouvelle du chapitre : `dbt build` parcourt le graphe dans l’ordre, construit chaque modèle puis le teste, et saute tout ce qui descend d’un échec. Un travail d’orchestration pour Boréal, c’est cette seule commande.

La troisième n’appartient pas à ce métier. Le Cours 06 de Data Engineering a monté Airflow, posé ses graphes et réglé le pool qui garde l’entrepôt ; un `dbt build --target prod` y entre comme une tâche parmi les autres, après le chargement de la `RAW` dont il dépend. Savoir cela suffit, et aller plus loin serait refaire le cours d’à côté.

Ce qui vous revient, en revanche, est de dire ce que la tâche attend et ce qu’elle promet : qu’elle tourne après le chargement, qu’elle est rejouable, et qu’un échec de test arrête ce qui vient après lui au lieu de le nourrir de chiffres faux. Cette promesse a une limite, qu’il faut savoir dire : dbt construit un modèle avant de le tester, donc le modèle fautif est déjà écrit quand son test tombe, et qui le lit directement le lit faux jusqu’à la correction. Le Cours 08 le mesure. La tâche doit donc finir en échec, pour que quelqu’un le voie, et ne rien lancer d’autre derrière elle. Ce contrat-là est analytique, pas technique.

Rappel — l’orchestration et l’intégration continue ne sont pas la même automatisation. Ici, on planifie l’exécution qui fait circuler la donnée. Au Cours 08, on vérifiera le code à chaque proposition de changement. L’une fait tourner la donnée, l’autre garde la qualité du code, et elles ne se remplacent pas.

11. Ce que trois mois changent

Labo : `Labo_07_AE.md`, étape 18.

Le Cours 06 s’est terminé sur un dossier ouvert. Le compresseur d’air numéro 2 a dérivé du 15 au 28 octobre 2025, 240 motoneiges ont été assemblées pendant cette fenêtre, et le taux de réclamation de cette cohorte, 2,5 pour cent, ne se distinguait pas de celui des autres. La conclusion honnête était qu’il n’y avait rien à conclure, et que les défauts d’un serrage insuffisant mettent des mois à paraître.

Trois mois ont passé, au sens propre : les lots de janvier, février et mars 2026 sont dans l’entrepôt. La même requête, inchangée, ne dit plus la même chose.

Le groupe	Véhicules vendus	Avec une réclamation de refroidissement	Taux
La cohorte exposée	237	34	14,35 %
Les autres motoneiges 2026	2 498	29	1,16 %

Douze fois plus, et le signal ne se voit que si l’on regarde la bonne cause : toutes causes confondues, les deux groupes affichent 27,0 et 19,2 pour cent, un écart qu’on n’aurait pas retenu.

La télémétrie, qui n’existait pas dans l’entrepôt au Cours 06, fournit un témoin indépendant, et c’est le meilleur de ce cours parce qu’il porte sa propre contre-épreuve.

© 2026 iAlexMG. Tous droits réservés.

Le code levé en roulant	Dans la cohorte, sur 95 connectées	Ailleurs, sur 7 549 connectées
`T310`, surchauffe	12, soit 12,6 %	84, soit 1,1 %
`T320`, liquide bas	17, soit 17,9 %	194, soit 2,6 %
`D240`, courroie	4, soit 4,2 %	496, soit 6,6 %
`E120`, batterie	2, soit 2,1 %	153, soit 2,0 %
`H510`, pression d'huile	1, soit 1,1 %	102, soit 1,4 %

Les deux codes du circuit de refroidissement séparent les groupes d'un facteur dix ; les trois autres ne les séparent pas du tout. Ce n'est plus une coïncidence qu'on constate, c'est un mécanisme : la cohorte diverge sur ce qu'un collier mal serré fait diverger, et sur rien d'autre.

Voilà ce que produit un entrepôt en régime permanent. Ce n'est pas un rapport de plus, c'est une question posée en octobre à laquelle on répond en mars, sans avoir rien réécrit, parce que les données ont continué d'arriver dans une forme que quelqu'un avait prévue.

Points clés à retenir

- Le régime permanent pose trois questions que le chantier ne pose pas : ce qui est neuf, ce qui a changé, et ce qu'on fait après une panne. La troisième est la plus importante, et sa réponse est l'idempotence.
- Un modèle incrémental repose sur trois pièces : `is_incremental()` ouvre le filtre, `{{ this }}` interroge la table déjà bâtie, `unique_key` déclare le grain. Le filtre sert la vitesse, la clé sert la justesse.
- Le filtre porte sur la date d'arrivée, jamais sur la date de l'événement. Un filtre sur l'événement perd les retardataires en silence : 47 sorties sur deux passes mensuelles, 2 559 si la passe suivait la cadence quotidienne de la source.
- Une fenêtre de reprise relit quelques jours à chaque passe. Sa largeur se choisit sur le plus long retard mesuré, ici 96 heures, et sa relecture n'est inoffensive que grâce à `unique_key`.
- On rend incrémental ce qui est gros et qui ne fait que grossir. Un fait d'accumulation, dont les lignes changent après coup, n'entre pas dans cette catégorie : 154 réclamations sur 3 348 sont réécrites par les lots de 2026, sans qu'un seul montant bouge.
- `--full-refresh` reconstruit un modèle de zéro, sans danger. Il ne touche pas aux instantanés, vérifié : `dbt snapshot` refuse l'option, et `dbt build --full-refresh` laisse les 205 versions en place. Ce qui compte, pour un instantané, est l'ordre dans lequel son histoire se monte.
- Une source semi-structurée porte des objets imbriqués, des listes et des champs qui changent de nom. Les trois se traitent au `STAGING`, à un seul endroit, pour que rien en aval ne voie de JSON.

© 2026 iAlexMG. Tous droits réservés.

- Une valeur qui sert de clé à deux faits mérite sa dimension conforme. Un `seed` est la bonne réponse pour un référentiel court et stable qu'aucune source ne livre.
- Deux environnements se séparent par une cible et une macro. Le code ne sait pas où il tourne, et c'est ce qui rend la production reproductible.
- Orchestrer, c'est planifier `dbt build --target prod`. Le quand appartient à Data Engineering ; ce qui vous revient est le contrat de la tâche.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_07_AE.md`. Faites-les après le labo : plusieurs énoncés supposent que le projet porte les trois mois de 2026.

© 2026 iAlexMG. Tous droits réservés.